MN692 Capstone Project

Secure Cloud Access and Network Segmentation for a Hybrid Enterprise

Final Report

Student's IDs and Names

MIT244011 - Raghava Reddy Siripuram

MIT241441 - Maruthi Avari

MIT244214 - Harshavardhan Kunta

MIT243787 - Nithin Yeleti

MIT242329 - Rajinikar Reddy Ummareddy

Industry Client

Industry/Company's Name: ThinkiX

Industry Contact's Name: Tharindu Athauda

Supervisor Name

MIT Supervisor's Name : Dr .Sana Jokhio

Trimester 3, 2026

School of Information Technology and Engineering



MELBOURNE
INSTITUTE OF TECHNOLOGY

Abstract

This project is for the implementation and evaluation of a practical Zero Trust security architecture for a hybrid enterprise environment between AWS and Azure, known as ThinkiX. Implementation focuses on unauthorised access, lateral movement and inconsistent security controls by implementing identity-aware application access, network segmentation, centralised monitoring and controlled response. The solution deployed combines Cloudflare Access and Cloudflare Tunnel to enable identity-based access to an application without opening inbound administrative access, AWS VPC segmentation to isolate Office, Management, Application and Third-Party zones, AWS Systems Manager Session Manager to enable IAM-based administration without SSH keys, and Microsoft Sentinel to collect CloudTrail and VPC Flow telemetry. An engineered network-flow features autoencoder Isolation Forest machine learning based pipeline for anomaly detection is combined with an Azure Function scoring service. The response path allows for adding or subtracting a Network ACL control from a response by using the response integration configured. Model artefacts are stored in Azure Blob Storage. Testing verified identity enforcement, segmentation controls, Sentinel ingestion, model scoring and controlled block/unblock response. The implementation itself shows the actual application of some of the core elements of Zero Trust, as well as the separation of implemented controls from other production capabilities like full cross-cloud federation, complete SASE traffic inspection, and SDN runtime orchestration.

Acknowledgement

We would like to thank Tharindu Athauda, Founder of ThinkiX, for his guidance, availability and constructive feedback throughout the design and implementation of this project. His feedback helped the team align the technical scope with the practical requirements of the organisation. We also acknowledge the support of our academic supervisor, Dr. Sana Jokhio, for guidance on research methodology, evaluation and technical reporting. Finally, we acknowledge the contribution of every team member to the architecture, cloud implementation, testing, documentation and final validation of the system.

Signature of Students:

MIT244011 - Raghava Reddy Siripuram

MIT241441 - Maruthi Avari

MIT244214 - Harshavardhan Kunta

MIT243787 - Nithin Yeleti

MIT242329 - Rajinikar Reddy Ummareddy

Report Submission Date: 22 September 2026

Table of Contents

Abstract 1
Acknowledgement 1
List of Figures 4
List of Tables 6
1. Introduction 7
2. Project Detailed Design 7
2.1 Implementation Overview 7
2.2 Software and Hardware Requirements 8
2.2.1 Software Requirements 8
2.2.2 Hardware Requirements 9
2.3 Weekly Activities and Team Responsibilities Plan 9
3. Project Implementation and Evaluation 10
3.1 Implementation 11
3.1.1 Infrastructure-as-Code Foundation 11
3.1.2 Five-Zone Network Segmentation 12
3.1.3 Application Zone and Identity-Based Administrative Access 13
3.1.4 ZTNA/SDP Deployment (Cloudflare Access) 14
3.1.5 SIEM Foundation and AI-Assisted Anomaly Detection 15
3.1.6 Controlled Automated Response and Network ACL Enforcement 18
3.1.7 Cloudflare Zero Trust and SASE-Aligned Controls 19
3.1.8 Identity Separation and Policy Automation 19
3.2 Testing and Troubleshooting 20
3.3 Results 21
3.4 Discussion and Analysis 22
3.5 Innovation 23
4. Conclusion, Limitation and Future Work 23
4.1 Conclusion 23
4.2 Limitation 24
4.3 Future Work 24
References 26
Glossary and Abbreviations 27
Appendices 28

Appendix I: Industry Client Information 28
Appendix II: Weekly Attendance Record with Industry Client/Supervisor 31
Appendix III: Summary of Updates for the Group Report Assessment 1 31
Appendix IV: Client and Supervisor Meeting Logbook 32
Appendix V: Source Codes or Other Relevant Information 33

List of Figures

Figure 1: Final hybrid Zero Trust architecture and monitoring-response flow

Figure 2: Terraform configuration for the deployed security infrastructure

Figure 3: Successful Terraform plan output

Figure 4: Azure Cloud Zone network

Figure 5: AWS subnets implementing the four AWS zones

Figure 6: Security group and Network ACL controls

Figure 7: Application-zone EC2 instance and Session Manager access

Figure 8: Cloudflare Access policy

Figure 9: Cloudflare Tunnel status

Figure 10: Cloudflare authentication and protected application access

Figure 11: Microsoft Sentinel workspace and AWS VPC Flow data

Figure 12: Feature dataset used for anomaly detection

Figure 13: Model training and threshold evidence

Figure 14: Azure Function scoring implementation and successful execution

Figure 15: Azure Blob Storage model artefacts

Figure 16: Sentinel KQL query used by the scoring function

Figure 18: NACL unblock/restore test

Figure 17: Response integration and NACL block test

Figure 19: Cloudflare Zero Trust/DLP configuration

Figure 20: IAM roles and IaC security-policy evidence

Appendix V Figure: Final source-code structure

List of Tables

Table 1: Software Requirements 9
Table 2: Hardware Requirements 9
Table 3: Weekly Activities 10
Table 4: Team Responsibilities Plan 10
Table 5: Testing and Troubleshooting 21
Table 6: Results 22
Table 7: Glossary and Abbreviations 28
Table 8: Attendance Record with Industry Client/Supervisor 31
Table 9: Summary of Updates 31
Table 10: Client and Supervisor Meeting Logbook 32

1. Introduction

ThinkiX is a Melbourne-based software services company operating a hybrid IT environment that includes on-premises resources and public cloud services. The project focuses on security challenges of remote access, cloud adoption and distributed infrastructure – all where traditional network-location-based trust can leave apps and internal resources vulnerable to unauthorised access and lateral movement. The original design provided a five-zone Zero Trust architecture and identified the key controls that needed to be implemented: identity-aware access, micro-segmentation, centralised security monitoring and policy automation.

The goal of implementation was to bring that design to fruition in a real, testable testbed. The ultimate deployment focused on the controls that could be deployed and tested in the project environment: Cloudflare Access and Tunnel for identity-aware application access; AWS VPC Segmentation for the Office, Management, Application and Third-Party zones; an Azure Cloud Zone; IAM-authenticated administrative access via AWS Systems Manager; Microsoft Sentinel for centralised telemetry; machine-learning anomaly scoring; and a controlled response path through AWS Network ACL enforcement. These controls are aligned to the Zero Trust principle, which is to verify access based on identity and context of resources instead of network location [1], [3], [19].

The project goals were: (1) move away from direct VPN-style application exposures to identity-aware access; (2) create explicit network boundaries with default-deny segmentation; (3) achieve centralised visibility of cloud activity and network flows; (4) identify abnormal network activity using a trained network anomaly model; and (5) validate a controlled response mechanism that blocks and restores a source host. Testing activities used to assess these goals included model validation and controlled response testing, Sentinel queries, and positive and negative access tests.

The detailed design and implementation plan is described in the remainder of the report, followed by a description and verification of the components deployed. The implementation, testing, results and analysis are presented in section 3, including machine-learning and response workflow. The outcomes, limitations and future work for the practical in Section 4 summarise the achievement.

2. Project Detailed Design

The detailed design maps out the Zero Trust goals into a series of actionable controls. The architecture decouples application access from network access, provides explicit separation of workload areas, and provides a telemetry-to-response pathway. This section provides an overview of implementation, resource needs and delivery plan.

2.1 Implementation Overview

The implementation is done by layering. Cloudflare Access passes authentication before reaching application in an outbound Cloudflare Tunnel on the access layer. At the workload layer, AWS VPC subnets and security groups are used to define four zones and Azure VNet is equivalent to the Cloud Zone. AWS Systems Manager Session Manager is an identity-based session service that is implemented at the administrative layer that does not expose SSH. AWS CloudTrail and VPC Flow Logs are ingested into Microsoft Sentinel at the monitoring layer. The anomaly pipeline fetches the recent flow data, creates behavioural features, and scores the traffic based on the behaviour by using an autoencoder and Isolation Forest. When the response path is triggered, the Network ACL that is configured is blocked or restored against a source host by invoking the webhook.

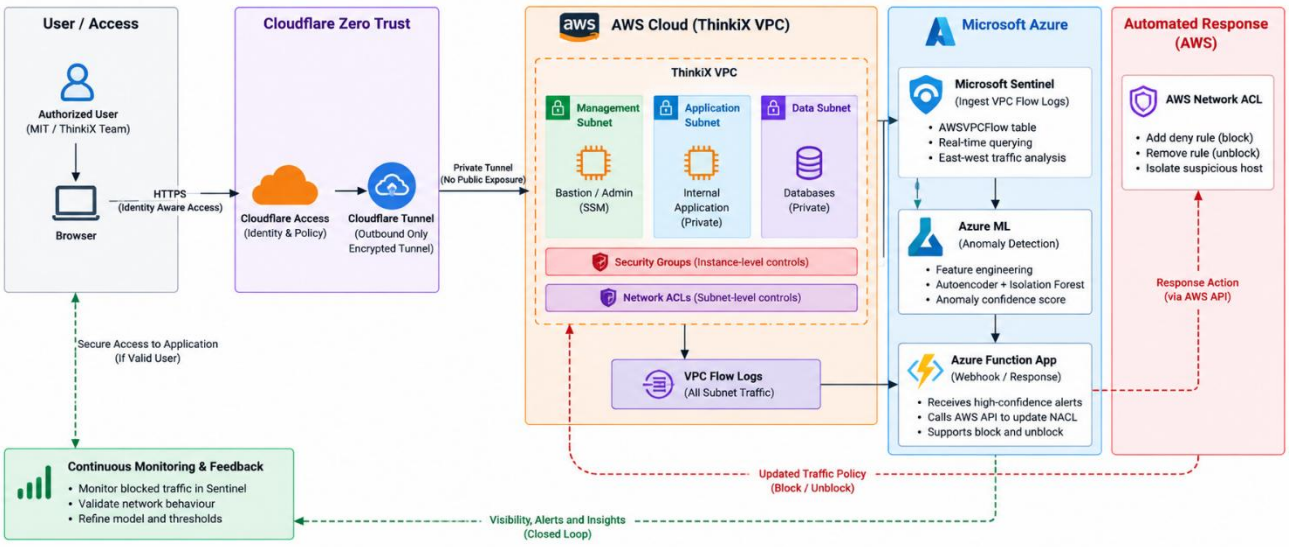


Figure 1: Final hybrid Zero Trust architecture and monitoring-response flow

Figure 1 should be read left to right as an enforcement & monitoring flow. Upon successfully proving their identity to Cloudflare Access, a user accesses an application that can be exposed by the tunnel without having an inbound application port that is publicly accessible. AWS then offers segmentation of workloads and administrative controls. VPC Flow records are recently loaded into Network telemetry which is used for behavioural scoring. The response path is isolated from the normal access path to allow a detected source to be dealt with without granting the detector wide network privileges. The layered approach is due to the principle of least privilege and explicit verification that Zero Trust places on its principle [9].

2.2 Software and Hardware Requirements

The implementation required cloud platforms, identity and access services, security monitoring, machine-learning libraries and Infrastructure-as-Code tooling. The selected resources were sufficient to reproduce the implemented prototype while keeping the environment within student-cloud cost constraints.

2.2.1 Software Requirements

Software	Purpose	Reference
Terraform	Infrastructure provisioning and version-controlled AWS/Azure configuration.	[4]
AWS IAM and Systems Manager	Identity-based administrative access and session auditing.	[2], [21]
Cloudflare Access and Tunnel	Identity-aware application access without direct inbound exposure.	[1], [3]
Microsoft Sentinel / Log Analytics	Centralised security telemetry and KQL-based investigation.	[7], [16]

Python, Pandas, NumPy, scikit-learn, TensorFlow	Feature engineering, autoencoder and Isolation Forest anomaly scoring.	[7], [10]
Azure Functions	Hosted scoring and response integration.	[7], [16]
Azure Blob Storage	Storage of trained model artefacts.	[15]
Checkov	Infrastructure-as-Code security policy validation before deployment.	[8]
AWS CLI / Azure CLI	Operational verification and controlled cloud administration.	[2]
Cloudflare Zero Trust / DLP controls	Access logging and SASE-aligned visibility.	[5], [11]

Table 1: Software Requirements

2.2.2 Hardware Requirements

Hardware / Resource	Purpose	Reference
AWS EC2 t3.micro instances	Application and Management-zone workloads for testing and traffic generation.	[4]
Cloudflare-connected endpoint	User-side access to the protected application.	[1], [3]
Developer workstation	Terraform, CLI, testing and report preparation.	—
Azure serverless resources	Function execution, model access and monitoring integration.	[16]

Table 2: Hardware Requirements

2.3 Weekly Activities and Team Responsibilities Plan

The project was delivered in an iterative manner allowing infrastructure, access control, monitoring and response to be tested as they became available. The following sequence maintains the original plan while communicating the implementation and final validation activities that took place:

Week	Phase	Key Activities
1–2	Discovery and baseline	Confirm AWS/Azure resources, review existing design, establish traffic and access baseline.
3–4	Design finalisation	Refine Zero Trust architecture, segmentation rules, identity model and monitoring workflow.
5–6	Core implementation	Provision AWS zones, deploy application instance, configure Cloudflare Access/Tunnel and Session Manager.

7–8	Monitoring and ML	Connect CloudTrail/VPC Flow Logs to Sentinel, generate East-West traffic, engineer features and train the model.
9–10	Integration testing	Deploy Azure scoring components, validate model retrieval and Sentinel queries, troubleshoot authentication and package dependencies.
11–12	Response and documentation	Validate controlled NACL block/unblock path, capture evidence, analyse results and prepare final documentation.

Table 3: Weekly Activities

The teams were responsible for architecture, ZTNA, AI/ML, SASE, network policy and reporting. All members were engaged in sprint reviews, testing, documentation and presentation exercises.

Team Member	**MIT ID**	**Responsibility**
Raghava Reddy Siripuram	MIT244011	ZTNA broker deployment and SDP configuration.
Nithin Yeleti	MIT243787	AI/ML SIEM integration and anomaly detection.
Maruthi Avari	MIT241441	SASE-aligned access and remote-access testing.
Harshavardhan Kunta	MIT244214	Network configuration and policy automation.
Rajinikar Reddy Ummareddy	MIT242329	Identity, access and compliance reporting.
All members	—	Sprint reviews, documentation, client presentation and validation.

Table 4: Team Responsibilities Plan

3. Project Implementation and Evaluation

In this section the design is documented and the evidence reported to assess the design. The evaluation method provides for separation between the configuration evidence and behavioural testing, which means that a control is not deemed effective simply because a configuration screen is provided. Thus, with each major control, a description and a verification method is provided as well as an observed outcome.

3.1 Implementation

3.1.1 Infrastructure-as-Code Foundation

Terraform was used as the primary Infrastructure-as-Code mechanism for provisioning and maintaining the cloud environment. The configuration included AWS networking, security groups, instances and supporting resources up to and including the resources needed on the Azure side for the monitoring and anomaly-detection workflow. The Terraform plan output was reviewed prior to making the changes to facilitate creation, modification and destruction of resources. This helped to minimize the manual configuration drift and allowed to establish a predictable infrastructure baseline [4].

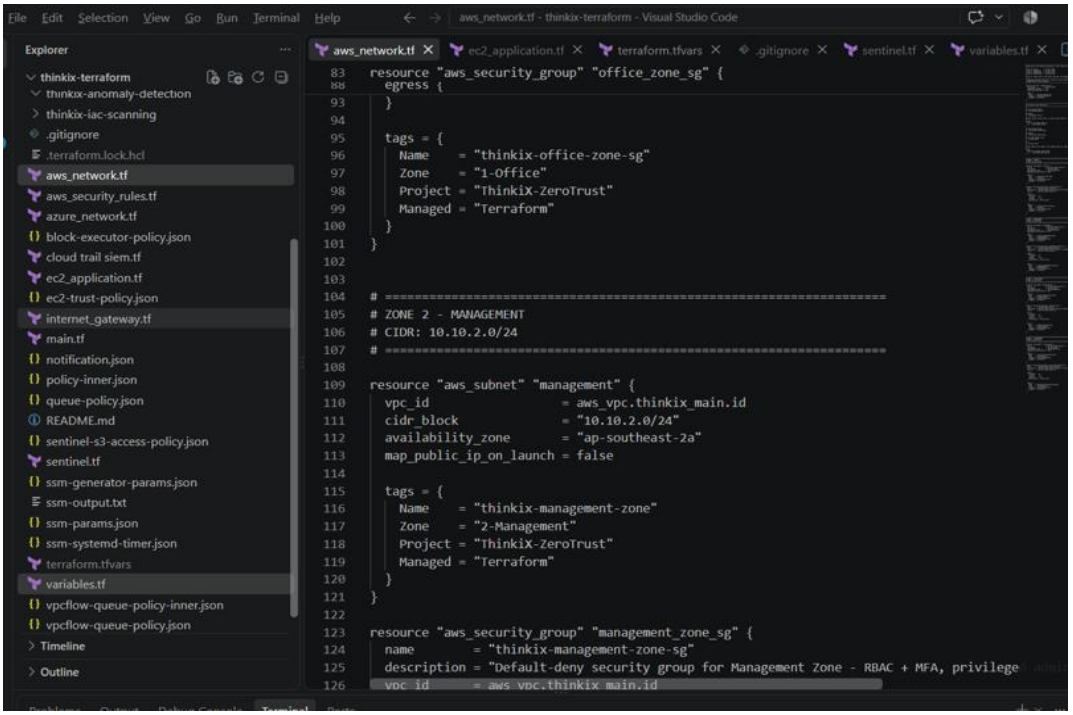


Figure 2: Terraform configuration for the deployed security infrastructure

The separation of human and automation identities was retained so that infrastructure provisioning did not depend on personal console credentials. This supports least-privilege and auditable administration [13], [21].



Figure 3: Successful Terraform plan output

3.1.2 Five-Zone Network Segmentation

Four AWS VPC subnets and an Azure Cloud Zone were used to implement the five-zone design. The AWS VPC uses the 10.10.0.0/16 address space, with Office at 10.10.1.0/24, Management at 10.10.2.0/24, Application at 10.10.3.0/24 and Third-Party at 10.10.4.0/24. This 10.20.0.0/16 address space is the Azure Cloud Zone. Security groups and Network ACL controls create clear traffic limits, and allowed application and administrative flows are restricted to the ports that the design needs. Reducing lateral movement paths is the basis of network segmentation [4], [6].

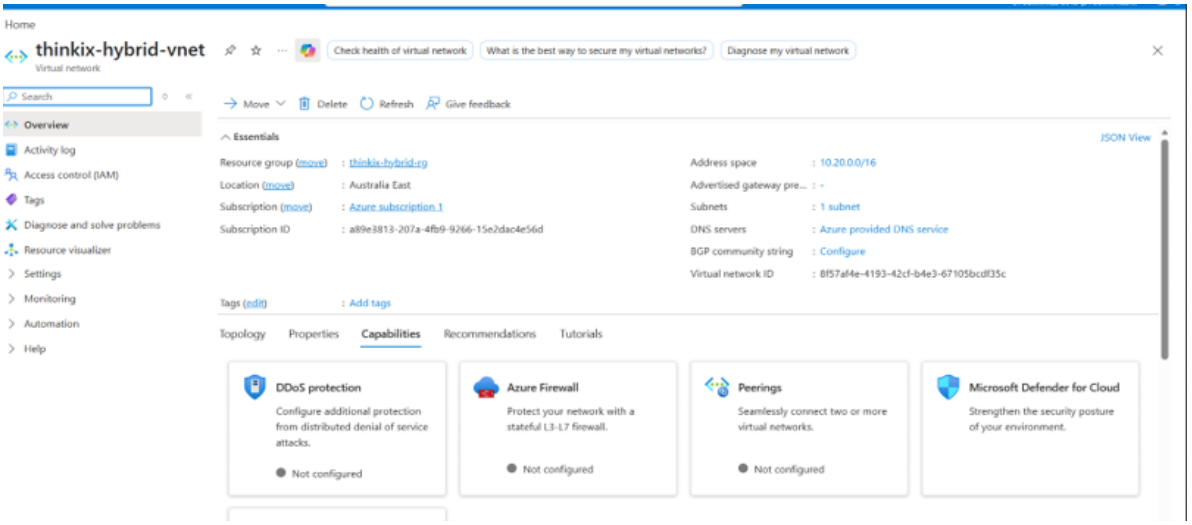


Figure 4: Azure Cloud Zone network

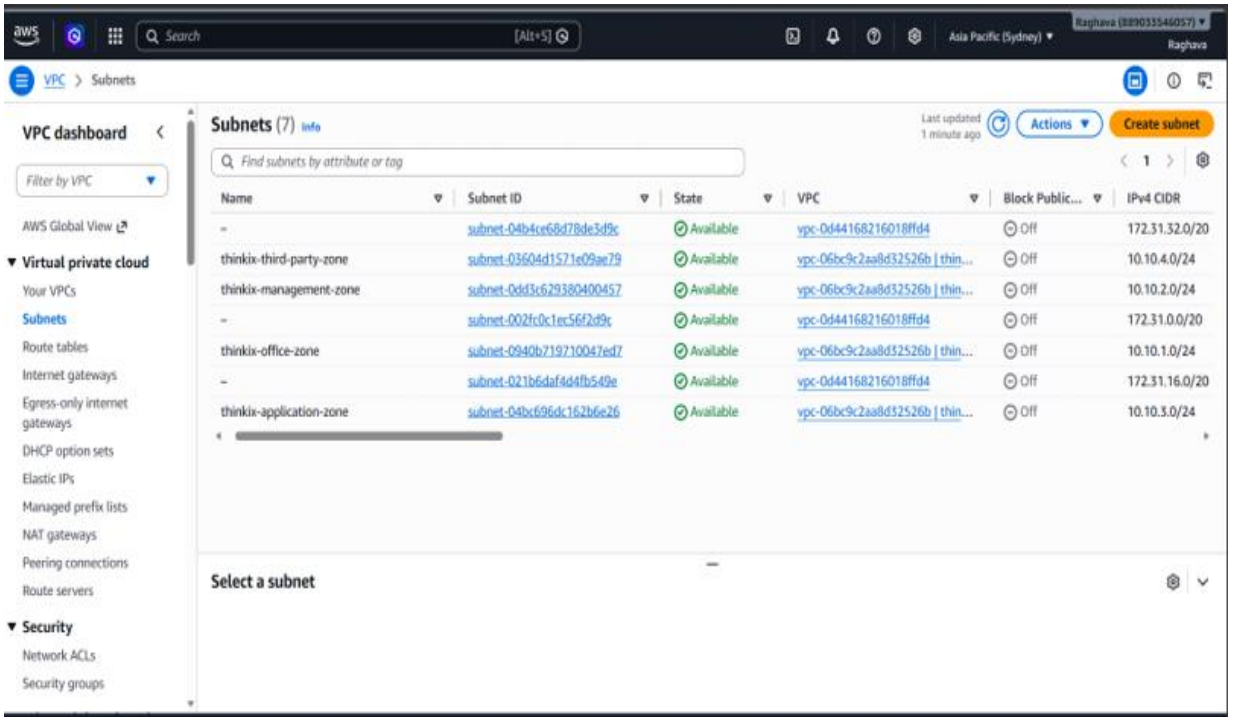


Figure 5: AWS subnets implementing the four AWS zones

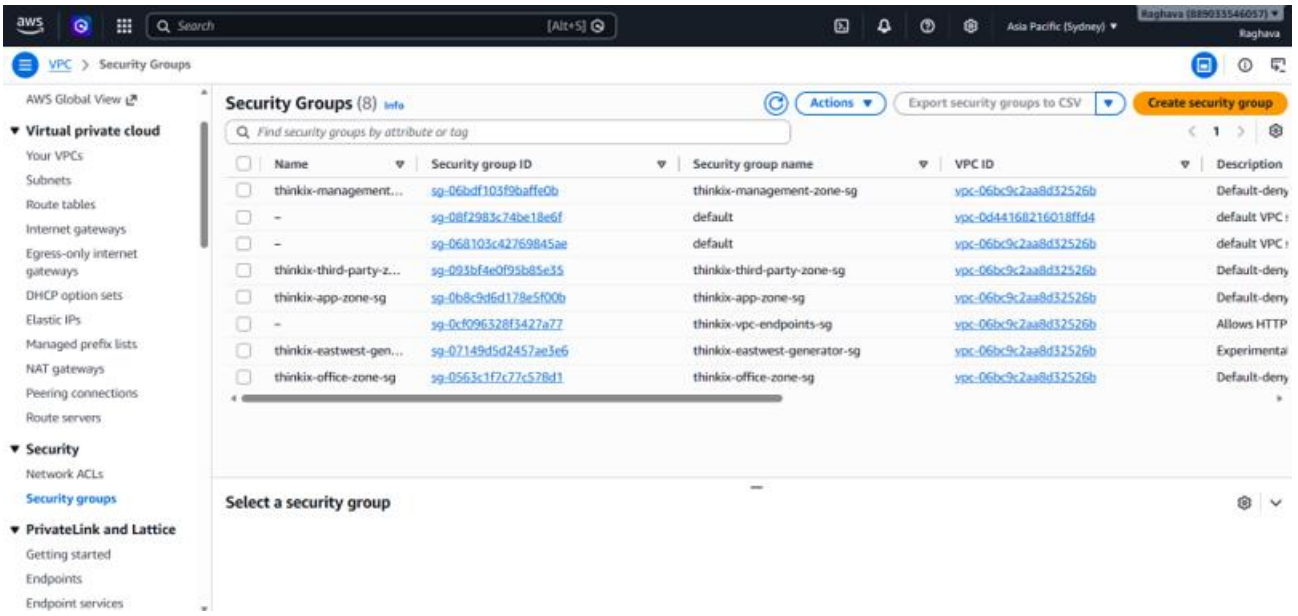


Figure 6: Security group and Network ACL controls

e Application-zone instance needs outbound connectivity to install software and to install the Cloudflare Tunnel agent, but has no inbound application/exposure as the trust mechanism. The decision to allow or deny access to the application is therefore separated from the security boundaries.

3.1.3 Application Zone and Identity-Based Administrative Access

The test application is running on a t3.micro EC2 instance in the Application zone. Administrative access is done with AWS Systems Manager Session Manager via IAM authentication. There is no SSH key pair needed and no inbound port 22 rule is applied. This provides a named, auditable administrative session and avoids dependence on long-lived SSH credentials. The design is consistent with least-privilege and identity-based administration [13], [21].

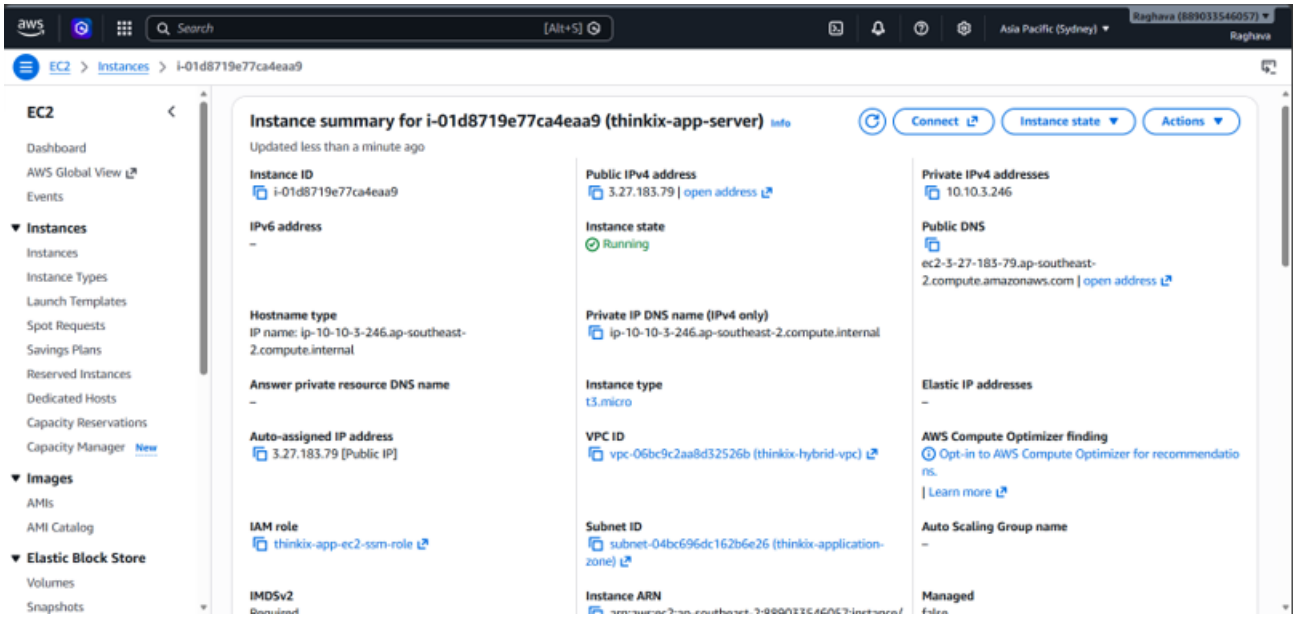


Figure 7: Application-zone EC2 instance and Session Manager access

3.1.4 ZTNA/SDP Deployment (Cloudflare Access)

Cloudflare Tunnel eliminates the need for an exposed inbound application port to connect the protected application to the public application domain. Cloudflare Access comes in front of the application and verifies identity before it sends the request. The configured policy restricts access to the approved student domain used for the demonstration. This changes the access decision from network reachability to an identity-aware application policy, which is the central practical Zero Trust control in the project [1], [3], [19].

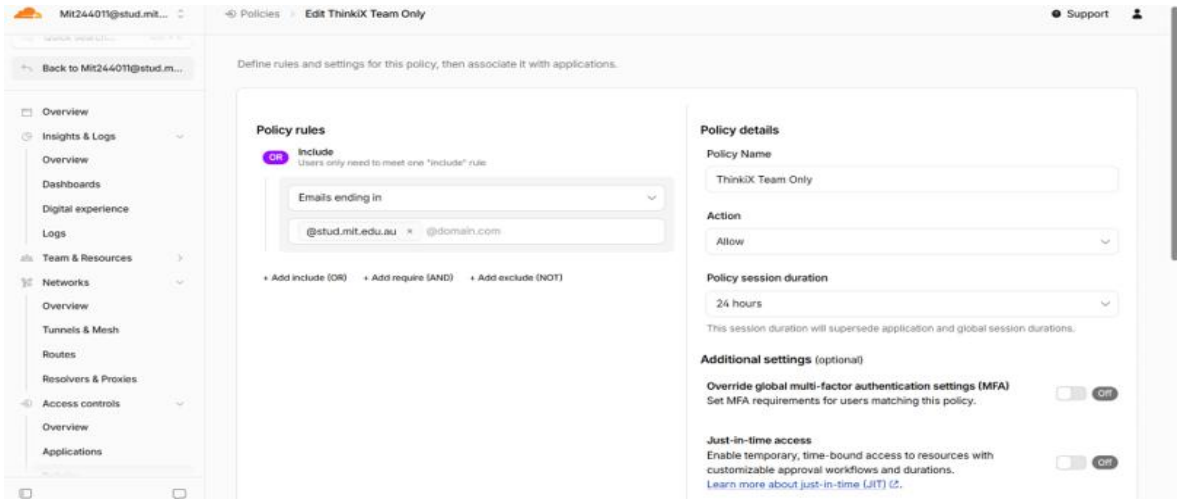


Figure 8: Cloudflare Access policy

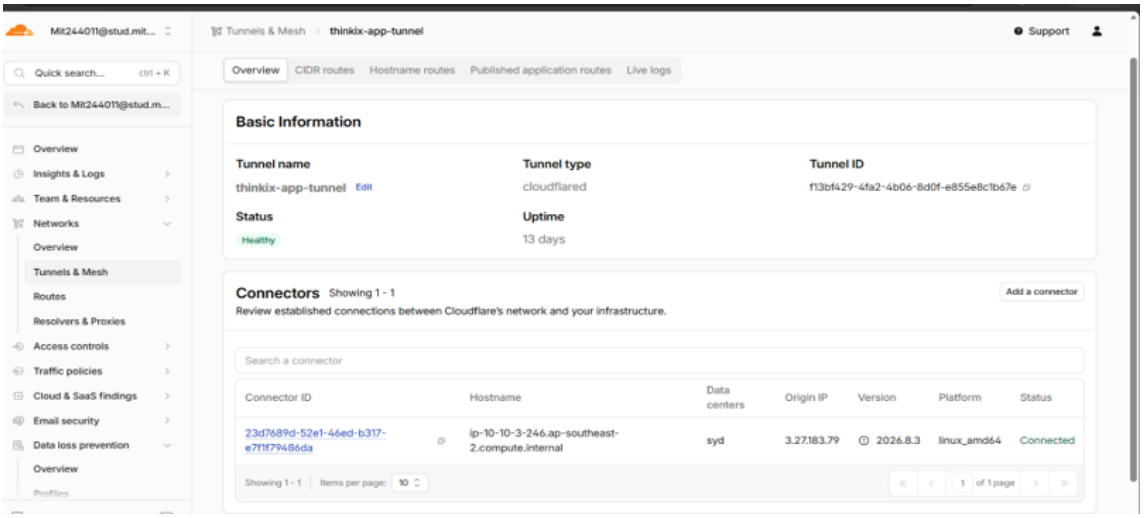


Figure 9: Cloudflare Tunnel status

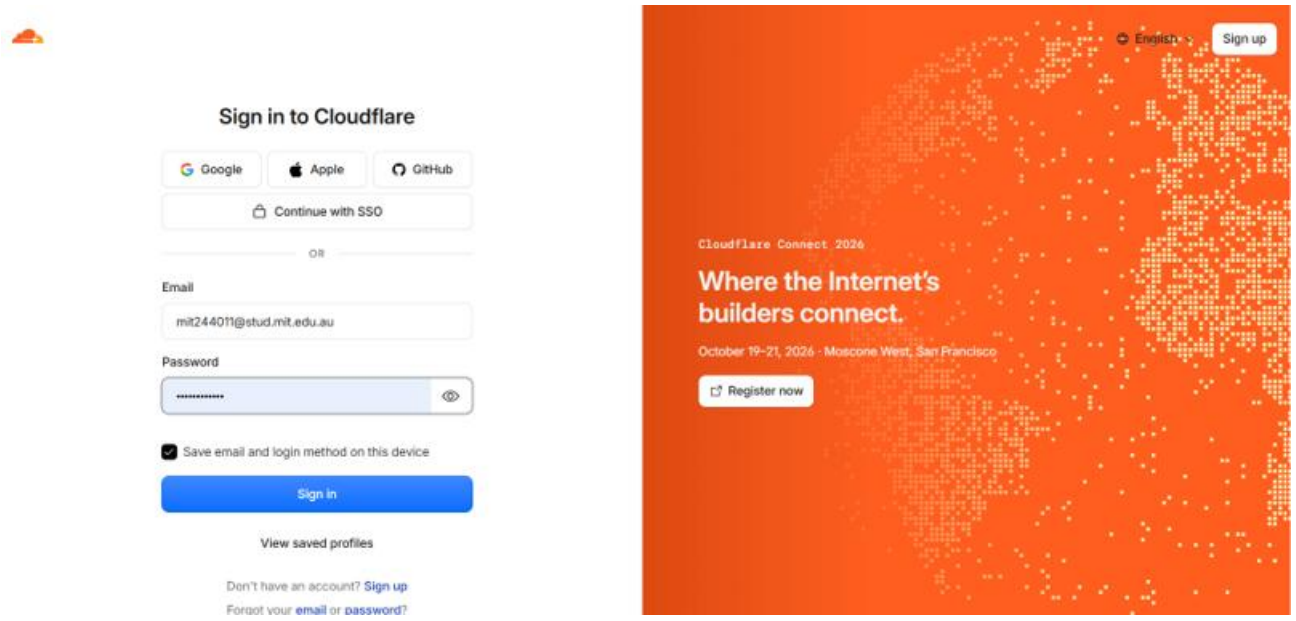


Figure 10: Cloudflare authentication and protected application access

Unauthenticated, authorised and unauthorised-domain requests were used to evaluate the access control. Each of these cases was separately seen in the Cloudflare access logs, and it was clear that the login screen was not a cosmetic feature, but indeed a forced access decision.

3.1.5 SIEM Foundation and AI-Assisted Anomaly Detection

Microsoft Sentinel and its Log Analytics workspace provide the central monitoring layer. AWS CloudTrail supplies API and account activity while AWS VPC Flow Logs supply network-flow telemetry. KQL queries were used to verify that the AWSVPCFlow table was getting live data, as well as the ability to isolate East-West traffic in the range 10.10.0.0/16. The monitoring component of the identified design needs is met by the centralised SIEM visibility, and the data source for behavioural analysis is the same.

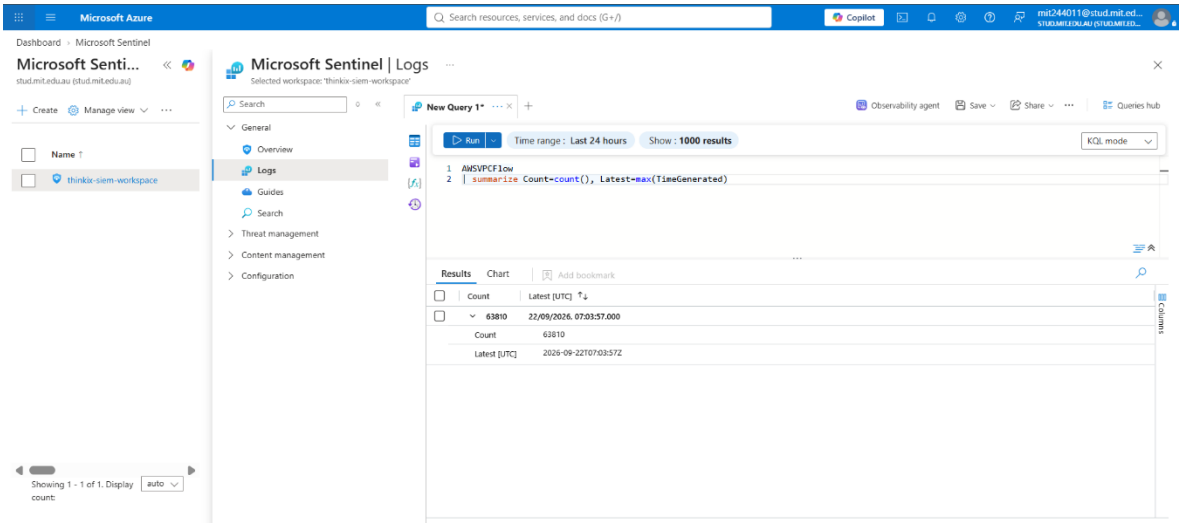
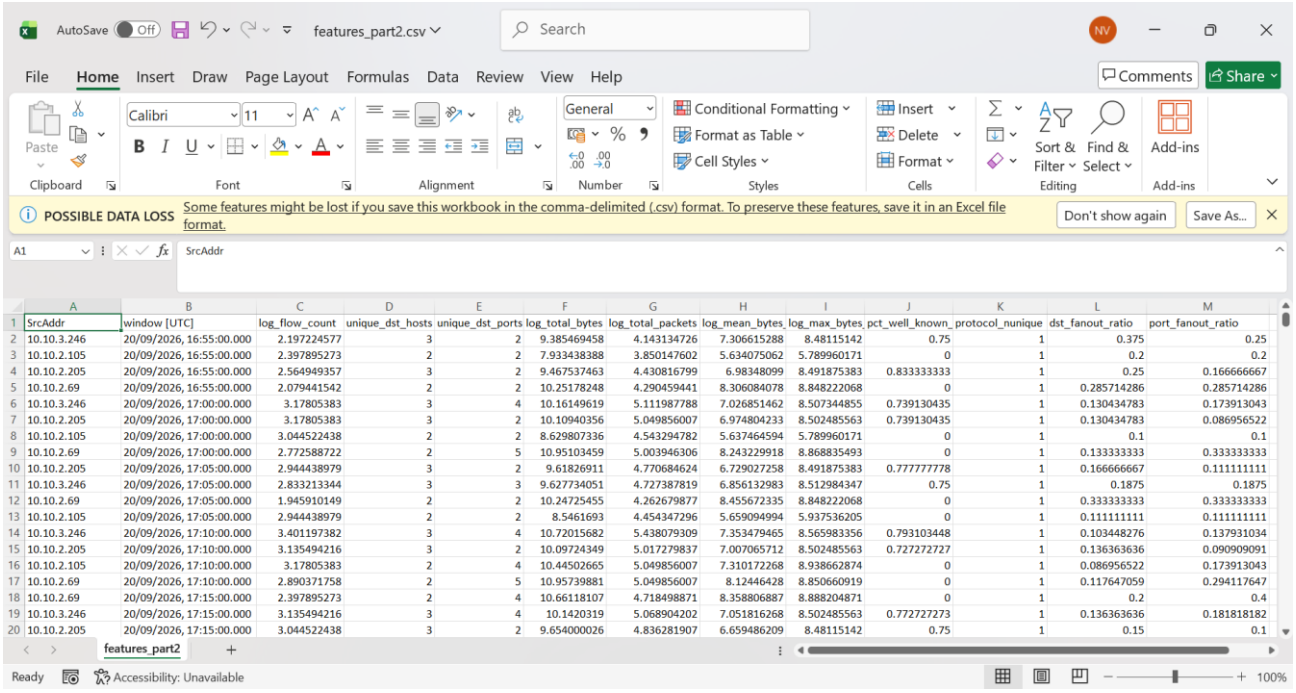


Figure 11: Microsoft Sentinel workspace and AWS VPC Flow data

A second EC2 instance in the Management zone created genuine East-West traffic towards the Application zone. The VPC Flow Logs thus gave the model developers a true baseline to develop the model from, beyond synthetic examples. A total of 3423 samples were processed from the traffic captured in the anomaly-detection dataset, based on 11 engineered network-behaviour features.



SrcAddr	window [UTC]	log_flow_count	unique_dst_hosts	unique_dst_ports	log_total_bytes	log_total_packets	log_mean_bytes	log_max_bytes	pct_well_known_	protocol_nunique	dst_fanout_ratio	port_fanout_ratio
10.10.3.246	20/09/2026, 16:55:00.000	2.197224577	3	2	9.385469458	4.143134726	7.306615288	8.48115142	0.75	1	0.375	0.25
10.10.2.105	20/09/2026, 16:55:00.000	2.397895273	2	2	7.933438388	3.850147602	5.634075062	5.789960171	0	1	0.2	0.2
10.10.2.205	20/09/2026, 16:55:00.000	2.564949357	3	2	9.467537463	4.430816799	6.98348099	8.491875383	0.833333333	1	0.25	0.166666667
10.10.2.69	20/09/2026, 16:55:00.000	2.079441542	2	2	10.25178248	4.290459441	8.306084078	8.848222068	0	1	0.285714286	0.285714286
10.10.3.246	20/09/2026, 17:00:00.000	3.17805383	3	4	10.16149619	5.111987788	7.026851462	8.507344855	0.739130435	1	0.130434783	0.173913043
10.10.2.205	20/09/2026, 17:00:00.000	3.17805383	3	2	10.10940356	5.049856007	6.974804233	8.502485563	0.739130435	1	0.130434783	0.086956522
10.10.2.105	20/09/2026, 17:00:00.000	3.044522438	2	2	8.629807336	4.543294782	5.637464594	5.789960171	0	1	0.1	0.1
10.10.2.69	20/09/2026, 17:00:00.000	2.772588722	2	5	10.95103459	5.003946306	8.243229918	8.868835493	0	1	0.133333333	0.333333333
10.10.2.205	20/09/2026, 17:05:00.000	2.944438979	3	2	9.61826911	4.770684624	6.729027258	8.491875383	0.777777778	1	0.166666667	0.111111111
10.10.3.246	20/09/2026, 17:05:00.000	2.833213344	3	3	9.627734051	4.727387819	6.856132983	8.512984347	0.75	1	0.1875	0.1875
10.10.2.69	20/09/2026, 17:05:00.000	1.945910149	2	2	10.24725455	4.262679877	8.455672335	8.848222068	0	1	0.333333333	0.333333333
10.10.2.105	20/09/2026, 17:05:00.000	2.944438979	2	2	8.5461693	4.454347296	5.659094994	5.937536205	0	1	0.111111111	0.111111111
10.10.3.246	20/09/2026, 17:10:00.000	3.401197382	3	4	10.72015682	5.438079309	7.353479465	8.565983356	0.793103448	1	0.103448276	0.137931034
10.10.2.205	20/09/2026, 17:10:00.000	3.135494216	3	2	10.09724349	5.017279837	7.007065712	8.502485563	0.727272727	1	0.136363636	0.090909091
10.10.2.105	20/09/2026, 17:10:00.000	3.17805383	2	4	10.44502665	5.049856007	7.310172268	8.938662874	0	1	0.086956522	0.173913043
10.10.2.69	20/09/2026, 17:10:00.000	2.890371758	2	5	10.95739881	5.049856007	8.12446428	8.850660919	0	1	0.117647059	0.294117647
10.10.2.69	20/09/2026, 17:15:00.000	2.397895273	2	4	10.66118107	4.718498871	8.358806887	8.888204871	0	1	0.2	0.4
10.10.3.246	20/09/2026, 17:15:00.000	3.135494216	3	4	10.1420319	5.068904202	7.051816268	8.502485563	0.772727273	1	0.136363636	0.181818182
10.10.2.205	20/09/2026, 17:15:00.000	3.044522438	3	2	9.654000026	4.836281907	6.659486209	8.48115142	0.75	1	0.15	0.1

Figure 12: Feature dataset used for anomaly detection

At the feature engineering step, data is aggregated into five minute per window/host observations. The engineered variables represent flow volume, destination diversity, port diversity, well-known-port usage, protocol diversity and fan-out or behaviour. The anomaly pipeline is a combination of an autoencoder reconstruction signal and an Isolation Forest score. A threshold of confidence is then set, before a response decision is made.

```
0.10.3.246 2026-09-03 18:25:00+00:00    0.828477    1.966160  -0.156955
0.10.2.205 2026-09-06 17:30:00+00:00    0.825312    6.058220  -0.150623
0.10.3.246 2026-09-06 17:30:00+00:00    0.825280    6.093655  -0.150560
0.10.3.246 2026-09-03 18:35:00+00:00    0.812072    2.037453  -0.124143
0.10.2.105 2026-09-03 18:30:00+00:00    0.807170    1.327330  -0.114341
0.10.2.105 2026-09-03 18:45:00+00:00    0.803387    0.819642  -0.106775
0.10.2.105 2026-09-03 18:50:00+00:00    0.803062    1.013328  -0.106125
0.10.2.105 2026-09-03 18:35:00+00:00    0.801853    0.817382  -0.103706
0.10.2.105 2026-09-03 18:55:00+00:00    0.801853    0.898556  -0.103706
0.10.2.105 2026-09-03 18:40:00+00:00    0.795968    1.099672  -0.091937
0.10.3.246 2026-09-03 18:20:00+00:00    0.794913    1.889852  -0.089826

================================================================================
UMMARY STATISTICS:
================================================================================
ean confidence:        0.2847
edian confidence:      0.2721
ax confidence:         0.8432
5th percentile:        0.3594
9th percentile:        0.5229
amples >= 0.85 threshold: 0 / 3,423 (0.00%)

rote full results to scored_real_data.csv
```

Figure 13: Model training and threshold evidence

The baseline scoring result produced no samples above the configured anomaly threshold during the clean baseline window. It is claimed to be a baseline-validation result and not necessarily a statement of the detector's detection and false-positive performance, which needs to be estimated with a longer and more diverse operational history [7], [10], [16].

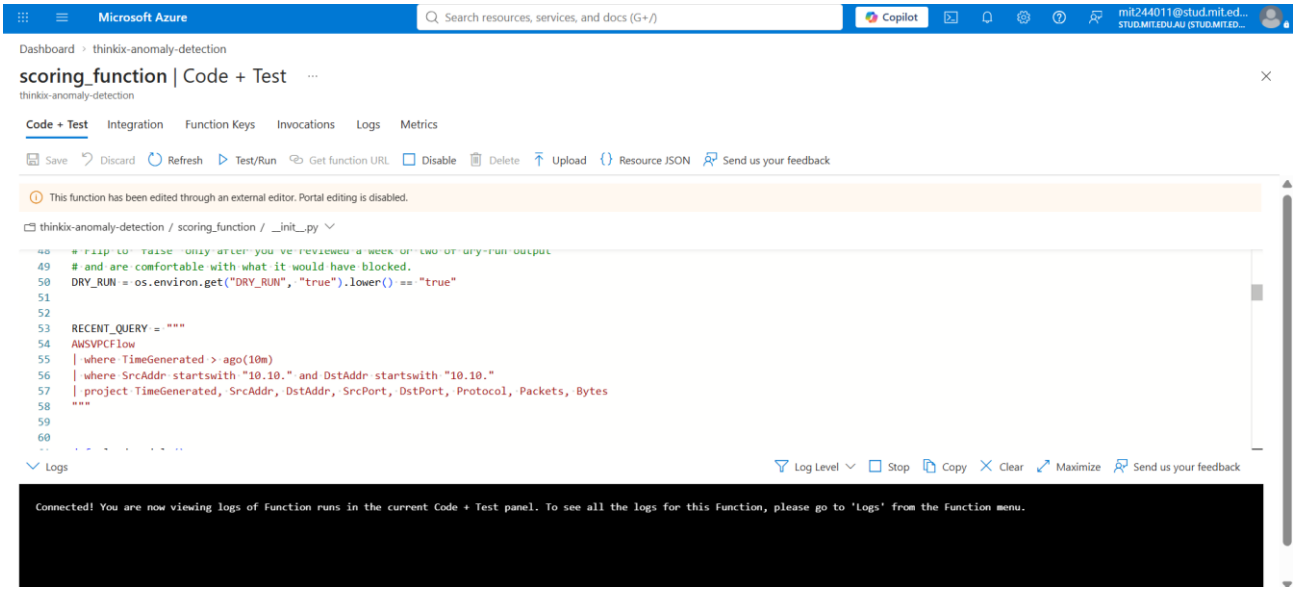


Figure 14: Azure Function scoring implementation and successful execution

The final implementation also handled Azure Function dependency and authentication issues observed in the previous implementation. The scoring function was locally tested with DefaultAzureCredential, fetched live samples from Sentinel, pulled the model artefacts from Azure

Blob Storage and was able to successfully score recent host/window samples. The same application package was created for the Linux Azure Function environment where the managed identity of the Function App was granted the necessary permissions to access the workspace and model artefacts of the read model.

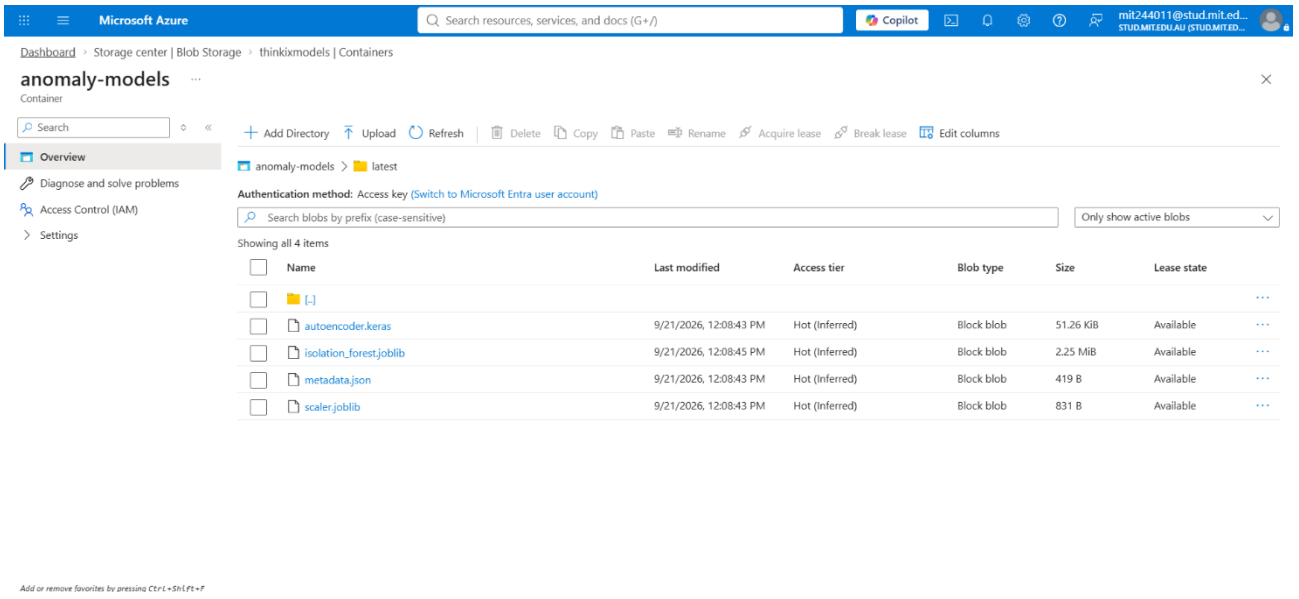


Figure 15: Azure Blob Storage model artefacts

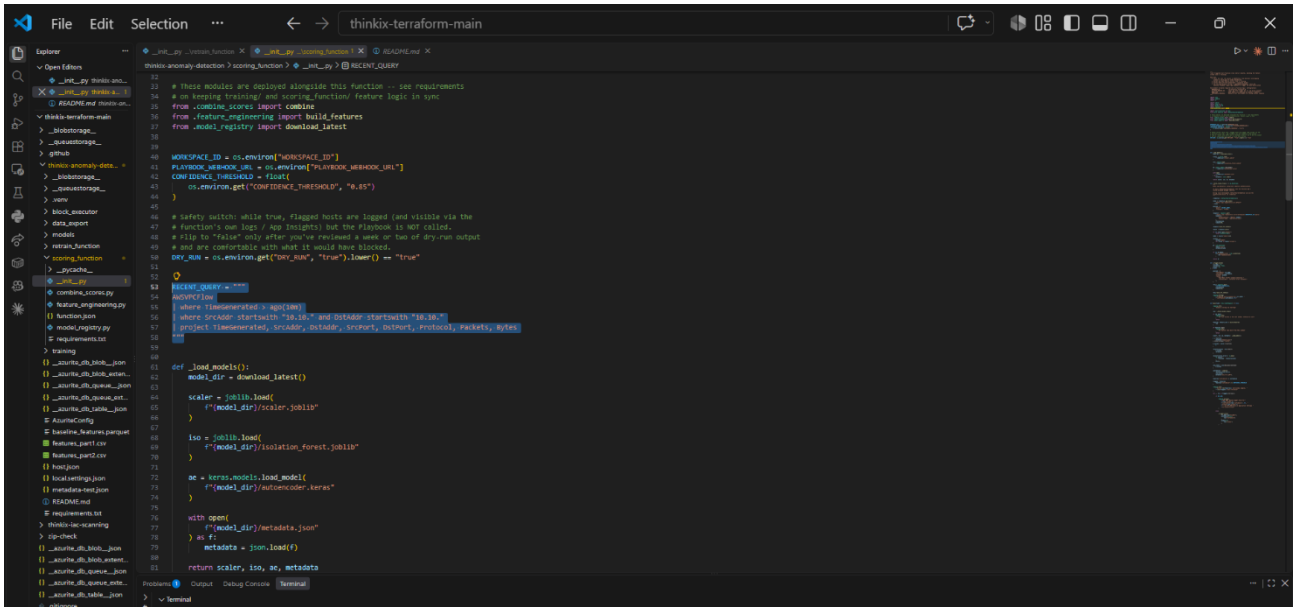


Figure 16: Sentinel KQL query used by the scoring function

3.1.6 Controlled Automated Response and Network ACL Enforcement

The response path specifies an AWS Network ACL action to perform on the result from the anomaly-scoring operation. The response payload contains the source host and a confidence value and reason for the action. The Network ACL mechanism provides an explicit enforcement point at the subnet boundary. The project therefore separates detection from enforcement: Sentinel supplies

telemetry, the scoring service evaluates behaviour, and the response mechanism applies the network control.

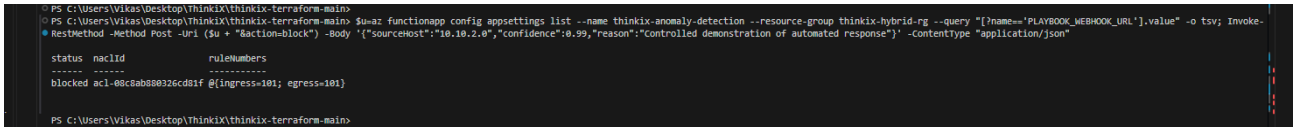


Figure 17: Response integration and NACL block test

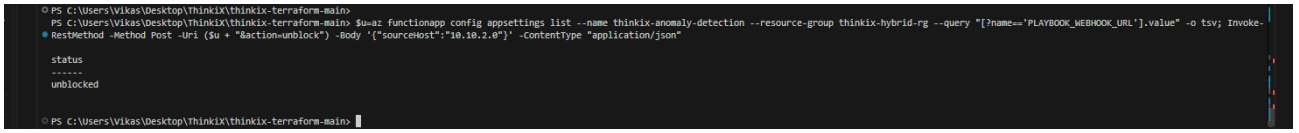


Figure 18: NACL unblock/restore test

The block and restore tests demonstrate that the enforcement mechanism can be applied and reversed against a specified source host. The final operational configuration needs to retain control and auditing over the response gate, in order to prevent a model decision directly changing unrelated network resources. This is based on the principle of least privilege policy enforcement [13], [19].

3.1.7 Cloudflare Zero Trust and SASE-Aligned Controls

The Cloudflare Zero Trust environment provides identity-aware access, access logging and a DLP profile. These controls offer access and visibility aspects associated with the project scope. Full enterprise SASE functionality, including universal endpoint traffic inspection and SD-WAN integration, was outside the implemented scope [5], [11].

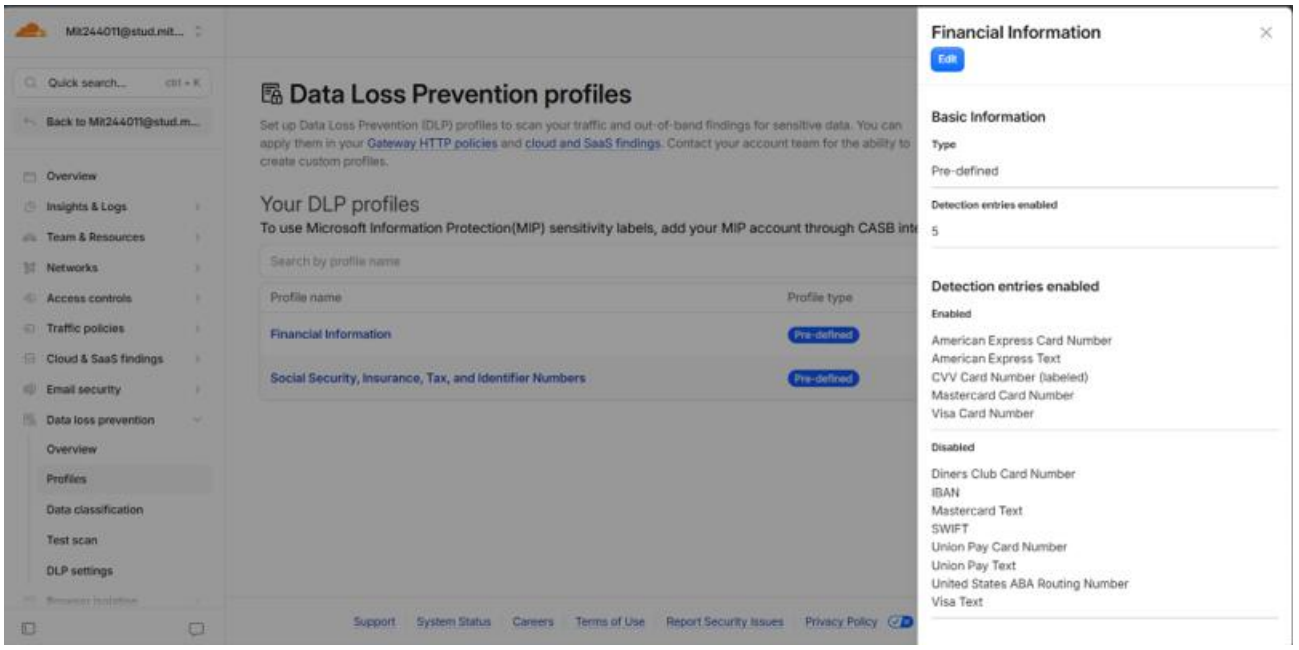


Figure 17: Cloudflare Zero Trust/DLP configuration

3.1.8 Identity Separation and Policy Automation

Human administrative identities were separated from automation identities used for infrastructure provisioning. IaC security checks are also used for the infrastructure pipeline to ensure security needs are checked prior to deployment. This minimises the chances that a change to the infrastructure is insecurely made without review, which helps fulfil the objective of secure development of the project [8], [15].

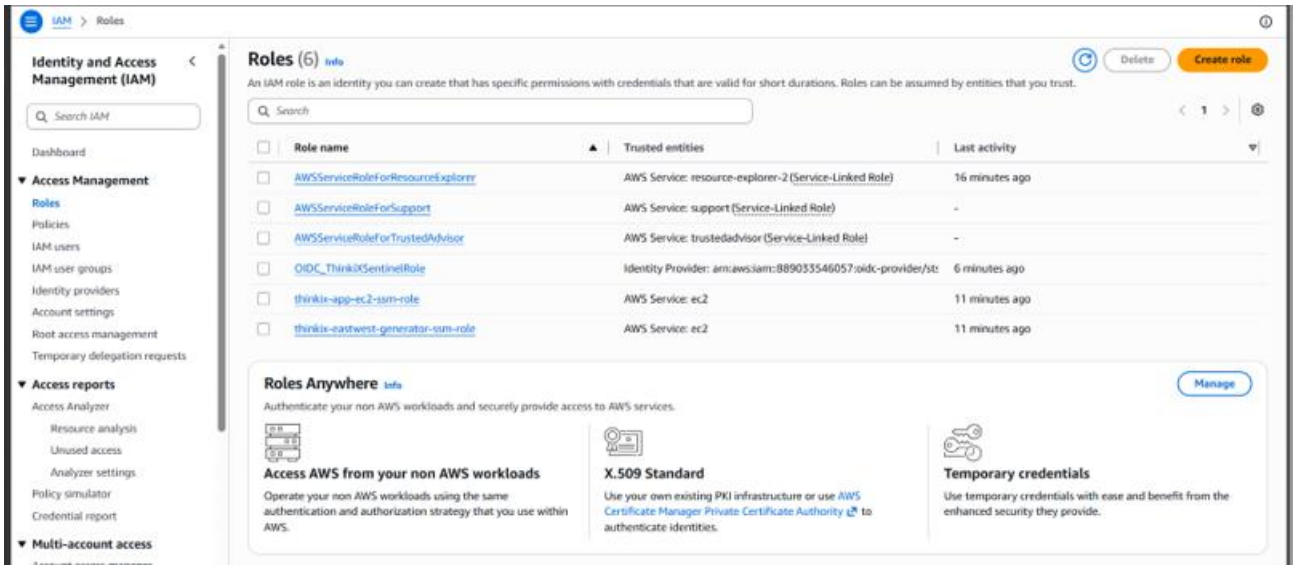


Figure 18: IAM roles and IaC security-policy evidence

3.2 Testing and Troubleshooting

Testing was done in a progressive manner as the system is composed of dependent cloud services. There were configuration tests conducted followed by functional tests, integration tests and controlled response tests. Troubleshooting focused on identifying the actual failure point rather than assuming that a configured resource was operational.

Test ID	Component	Test	Observed Result	Status
TC-01	Cloudflare Access	Unauthenticated request to protected application	Redirected to identity verification.	Pass
TC-02	Cloudflare Access	Approved-domain authentication	Application access granted after identity verification.	Pass
TC-03	Cloudflare Access	Non-approved-domain authentication	Access denied by policy.	Pass
TC-04	AWS segmentation	Review of security-group/NACL rules	Only explicitly permitted traffic paths are allowed; default-deny controls remain in place.	Pass
TC-05	AWS administration	SSH attempt to Application zone	No inbound SSH rule is exposed; administration is performed through Session Manager.	Pass
TC-06	Sentinel	Query AWSVPCFlow for recent telemetry	Live flow records returned from the	Pass

			Sentinel workspace.	
TC-07	ML scoring	Score recent Sentinel host/window samples	Scoring completed successfully and returned host/window results.	Pass
TC-08	Model storage	Download model artefacts from Blob Storage	Required model files were retrieved successfully.	Pass
TC-09	Response integration	Controlled block request for source 10.10.2.6	Response returned blocked and NACL state reflected the deny action.	Pass
TC-10	Response integration	Controlled unblock request for source 10.10.2.6	Response returned unblocked and the temporary deny action was removed.	Pass

Table 5: Testing and Troubleshooting

There were a few implementation issues that were dealt with and solved. The Azure scoring path failed to run as there were missing dependencies in the deployed environment, so remote build configuration was enabled and the deployment package was rebuilt. The initial response from Authentication was an invalid Log Analytics token, caused by the local Azure CLI credential being stale; the Azure CLI session was refreshed and another test of the REST query resulted in a valid token to the correct audience. Then the scoring implementation was updated to utilize DefaultAzureCredential to enable local testing using authenticated Azure CLI and hosted testing using the Function App managed identity. Model-download failures also occurred because of a malformed Blob Storage endpoint, which was fixed to the canonical storage account endpoint. These steps were crucial to the troubleshooting process since they proved the monitoring and ML chain need to be validated as a whole system, not as separate components.

3.3 Results

The final results are summarised below. The results separate out implemented control measures from more general design goals to avoid over-claiming what was shown.

Evaluation Area	**Result / Evidence**
Identity-aware access	Unauthenticated users were challenged; approved identities were granted access; non-approved identities were denied.
AWS network segmentation	Four AWS zones were deployed with explicit security boundaries and default-deny controls.
Administrative security	Application-zone administration used IAM-authenticated Systems Manager sessions rather than exposed SSH keys/port 22.
Centralised monitoring	AWS CloudTrail and AWS VPC Flow Logs were visible in Microsoft Sentinel through live KQL queries.

ML baseline	3,423 processed flow samples were used for model training and baseline validation; the clean baseline produced no threshold exceedances.
Live scoring	The scoring pipeline successfully queried Sentinel, loaded model artefacts from Blob Storage and completed scoring of recent host/window samples.
Response enforcement	Controlled block and unblock actions against a specified source host returned the expected status and were reflected in the AWS Network ACL control.
Cross-cloud integration	AWS and Azure resources were deployed, but private AWS-Azure network connectivity was not established; therefore full cross-cloud traffic policy validation remains outside the demonstrated scope.
SASE scope	Cloudflare Access, logging and DLP configuration were implemented; full WARP/SD-WAN traffic inspection was not demonstrated.
Runtime SDN	IaC policy validation was implemented; a dedicated runtime SDN controller for dynamic traffic steering was not implemented.

Table 6: Results

The overall security path of the project is working at the prototype level: identity is assessed before access to an application, workload traffic is separated, the telemetry is centralised, network activity can be graded, and a controlled response can influence a network boundary. The remaining gaps are mostly related to production hardening and/or limited breadth and not to the lack of the core prototype controls.

3.4 Discussion and Analysis

The most important takeaway is the adoption of Zero Trust as an access-control behaviour and not a label for architecture. This protected Application is not dependent on a network location. Requests have to go through Cloudflare Access and the result varies based on the identity provided. The same application URL thus returns different responses from unauthenticated, authorised and unauthorised users. It is a similar approach to the Zero Trust principle of "explicit verification" described in literature [1], [3], [19].

The network segmentation layer complements the identity decision. In addition to being able to navigate to the Application via the approved "pathway", AWS security groups and Network ACLs offer different "traffic boundaries". This defence-in-depth approach is crucial because even if identity-aware application access is used to stop application access, lateral network activity is not necessarily stopped. The default deny rules and explicit allow paths are an example of how the micro-segmentation approach mentioned in the literature was applied [4], [6].

The Sentinel and ML modules build on the preventive controls architecture to provide monitoring and response. The data-availability issue was resolved in the previous deployments by the introduction of a second Management-zone workload, which created real East-West traffic. The 3423 samples in the training set are thus not just synthetic but real examples of network behaviour. The live scoring test showed that the feature-engineering and model-serving path can be used to eat up real-time data from Sentinel. The clean baseline should be considered with care, however, because there was no intentional malicious traffic and it was not operated for long enough periods of time to reliably determine the detection rate or false-positive rate [7], [10], [16].

The response path is a visual representation of the end concept of enforcement at a controlled scope. A source host can be added to the response integration and then be denied and removed from a Network ACL deny action. This establishes a clear line from telemetry to analysis to enforcement. The response identity should be kept to a narrow focus for production use, and the actions logged, with a human approval or confidence policy being suitable for high impact controls. These are aligned with least privilege and risk-driven Zero Trust principles [13], [18].

The project also illustrates the importance of implementation discipline. The first credential exposure in the tunnel testing revealed that insecure test tooling can still be a threat to a Zero Trust production architecture. The team isolated the service, took off the exposure and placed the test application to dedicated cloud infrastructure. This experience is consistent with the overall security need of uniformity of controls throughout development and operation [8], [14], [15].

3.5 Innovation

It is a single bundle of identity-aware application access, infrastructure segmentation, centralised telemetry, machine-learning scoring and network enforcement that can be demonstrated in a single security workflow. These technologies are meshed together via specified interfaces: Cloudflare Access secures Application access, AWS control provides workload boundaries, Sentinel provides telemetry layer, Azure Functions provides orchestration and scoring, Blob Storage provides storage of model artefacts and AWS Network ACLs provides a concrete enforcement point.

The second innovation was real East-West traffic used to build up the baseline for anomaly detection. This extra instance of Management-zone produced the information necessary to train and test the model using real network activity. To minimize the use of static credentials and manual configuration, the project also deployed Infrastructure-as-Code checks and managed identities. These decisions are in accordance with the principles of secure software development and federated identity [13], [14], [15].

Finally, reversible enforcement of the response path was shown. The fact that it was possible to cut off a particular source and then return to the status quo allowed them to verify the response mechanism without destroying the experiment. This provided a practical bridge between anomaly analytics and network security enforcement.

4. Conclusion, Limitation and Future Work

The last part summarizes the results of the proof of concept, formalizes the range of evidence and outlines the additional engineering work necessary to take the validated prototype to a product-ready hybrid security platform.

4.1 Conclusion

The project successfully transformed the core aspects of the proposed Zero Trust design to a working prototype for a hybrid-cloud-based security solution. Cloudflare Access and Tunnel were used to enable identity-aware access, IAM-authenticated Systems Manager sessions were used for administrative access and AWS telemetry was consolidated in Microsoft Sentinel, which was the workload zone used for centralising AWS telemetry. The anomaly-detection workflow was trained on real captured traffic and the final scoring pipeline was integrated with live Sentinel data and model artefacts stored in Azure Blob Storage.

The controlled response workflow wrapped up the monitoring to enforcement path of the architecture at prototype scope. A source host may be passed to the response integration, a Network ACL deny action may be applied and the action may then be removed. The result is more of a

practical implementation of identity, segmentation, monitoring/control of response, as opposed to a purely conceptual Zero Trust diagram.

The project does not claim that the environment is a complete production implementation of every component proposed in the original MN690 design. Cross-cloud private connectivity, full federated SSO, complete SASE traffic inspection, runtime SDN control and formal adversarial validation remain outside the demonstrated scope [12]. This distinction is important because Zero Trust is an architectural approach that requires continuous verification and operational governance rather than a single product deployment [18], [19].

4.2 Limitation

The first limitation is evaluation breadth. The anomaly model was tested against a clean real traffic baseline, however it was not possible to create a statistically representative set of labeled attacks. Thus, the current results are indicative of Pipeline functionality and baseline behaviour, but are not defensible in terms of production detection rate or false positive rate.

The second one is the cross-cloud connectivity. AWS and Azure were not integrated with each other via a site-to-site VPN, VPC/VNet peering or similar private connection. End-to-end cross-cloud segmentation policies were not applied, so the Cloud Zone can be documented and monitored [20].

The third limitation is identity federation and privileged access management. The environment that has been implemented is separating human and automation identities, however full cross-cloud SAML/OIDC federation and just-in-time PAM were not implemented. Standing permissions are thus carried over into parts of the administrative model. A dedicated PAM solution can be added in the future within the project's cost & integration boundaries [21].

The fourth limitation is SASE scope. Cloudflare Access, access logging and a DLP profile were implemented, but the project did not deploy complete WARP-based endpoint traffic inspection or SD-WAN integration. The resulting implementation should therefore be described as SASE-aligned rather than as a complete SASE deployment [5], [11].

The fifth limitation is adversarial testing. There was no formal red team evaluation with a specific attack plan, although functional or integration testing was carried out, including controlled response verifications. Hence, it is not possible to make a strict claim that the system has passed an independent zero-lateral-movement test [22], [23] and only the observed outcomes of the tests in the project can be reported.

Last but not least, the architectural design and extent of continuous testing were affected by cloud student-account and cost constraints. Carefulness was needed in choosing the resources and making cost-aware choices, as Azure for Students is not available on all resources and AWS Free Tier only offers a limited selection of resources [24], [25].

4.3 Future Work

Future development should include expanding and mature the validated pipeline with greater breadth and production. First, the baseline needs to be expanded to include more workloads and Azure network telemetry to ensure that the model will have a broad spectrum of "normal" behaviour. A labelled attack dataset should then be introduced to measure detection rate, false-positive rate, alert latency and containment time.

Second, the anomaly response should be hardened for operational use. The scoring service should have a very limited scope for the managed identity, all response actions be fully logged, and a confidence/approval policy be set to differentiate the low-risk controls from the high-impact network changes. Unattended enforcement should not be enabled until a longer observation period.

Third, AWS-Azure private connectivity, and federated identity, should be added to enable the Cloud Zone to be part of the same end-to-end policy model. This would enable the cross cloud micro-segmentation to be assessed, not inferred from individual address spaces [20].

Fourth, the SASE layer should be extended with appropriate endpoint traffic inspection and SD-WAN capabilities, and a dedicated PAM solution should be evaluated for time-limited administrative privileges [5], [11], [21]. Finally, a formally scoped penetration test should be conducted after these extensions to evaluate lateral movement and policy bypass under adversarial conditions [22], [23].

References

[1] S. Q. Ajmi and S. S. Majeed, “Securing cloud computing networks using secure access protocols on zero trust architecture,” International Journal of Computing and Artificial Intelligence, vol. 6, no. 1, pp. 159–163, 2025.

[2] S. Haji, “A Hybrid Model for Information Security Risk Assessment,” International Journal of Advanced Trends in Computer Science and Engineering, vol. 8, no. 1, pp. 100–106, 2019.

[3] H. Rehan, “Zero-Trust Architecture for Securing Multi-Cloud Environments,” Cybersecurity and Network Defence Research, vol. 2, no. 2, pp. 236–272, 2022.

[4] S. Jorepalli, “Exploring network segmentation techniques for enhanced security in multi-tenant cloud environments,” International Journal of Innovation Studies, vol. 7, no. 2, pp. 41–50, 2023.

[5] T. Chande and D. H. Maniar, “Secure Access Service Edge (SASE): A Comprehensive Review of Architecture, Security Capabilities, and Enterprise Adoption,” International Journal of Innovative Research in Science Engineering and Technology, vol. 15, no. 3, p. 2641, 2026.

[6] L. Harris, “Comparing Zero Trust Network Segmentation vs. Traditional Network Security: Why Organizations Must Adapt,” ResearchGate, 2025.

[7] T. K. Adenekan, “Leveraging Artificial Intelligence for Enhanced Cybersecurity in Hybrid Cloud Environments,” ResearchGate, 2020.

[8] V. V. Baligodugula, “An Overview of Secure Network Segmentation in Connected IIoT Environments,” Scifinti, 2024.

[9] J.-H. Park, S.-C. Park, and H.-Y. Youm, “A proposal for a zero-trust-based multi-level security model and its security controls,” Applied Sciences, vol. 15, no. 2, p. 785, 2025.

[10] Z. Senan Mahmod Attar Bashi and S. Senan, “A comprehensive review of zero trust network architecture (ZTNA) and deployment frameworks,” International Journal on Perceptive and Cognitive Computing, vol. 11, no. 1, pp. 148–153, 2025.

[11] P. Raghavan, “The Rise of Secure Access Service Edge (SASE) in Cloud Performance and Security,” International Journal of Computer Technology and Electronics Communication, vol. 3, no. 1, pp. 2012–2016, 2020.

[12] M. Uddin, S. Mukherjee, H. Chang, and T. V. Lakshman, “SDN-based multi-protocol edge switching for IoT service automation,” IEEE Journal on Selected Areas in Communications, vol. 36, no. 12, pp. 2775–2786, Dec. 2018, doi: 10.1109/JSAC.2018.2871325.

[13] S. Potluri, “A zero trust-based identity and access management framework for cross-cloud federated networks,” International Journal of Emerging Research in Engineering and Technology, vol. 5, no. 2, pp. 28–40, Apr. 2024, doi: 10.63282/3050-922X.IJERET-V5I2P104.

[14] H. Kim, Y. Kim, and S. Kim, “A study on the security requirements analysis to build a zero trust-based remote work environment,” arXiv:2401.03675, Jan. 2024, doi: 10.48550/arXiv.2401.03675.

[15] I. Coston, K. D. Hezel, E. Plotnizky, and M. Nojoumian, “Enhancing secure software development with AZTRM-D: An AI-integrated approach combining DevSecOps, risk management, and zero trust,” Applied Sciences, vol. 15, no. 15, p. 8163, 2025.

[16] I. Mail, “The future of SIEM in a machine learning-driven cybersecurity landscape,” Turkish Journal of Computer and Mathematics Education, vol. 14, no. 03, pp. 1309–1314, 2023.

[17] H. Doorewaard, G. Van Hootegem, and R. Huys, “Team responsibility structure and team performance,” Personnel Review, vol. 31, no. 3, pp. 356–370, 2002.

[18] N. Papakonstantinou, D. L. Van Bossuyt, J. Linnosmaa, B. Hale, and B. O’Halloran, “A zero trust hybrid security and safety risk analysis method,” Journal of Computing and Information Science in Engineering, vol. 21, no. 5, p. 050907, 2021.

[19] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[20] M. Tsai, S. Lee, and S. W. Shieh, “Strategy for Implementing of Zero Trust Architecture,” IEEE Transactions on Reliability, vol. 73, no. 1, pp. 93–100, Mar. 2024, doi: 10.1109/TR.2023.3345665.

[21] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. A. Baig, and R. Doss, “Zero Trust Architecture (ZTA): A Comprehensive Survey,” IEEE Access, vol. 10, pp. 57143–57179, 2022, doi: 10.1109/ACCESS.2022.3174679.

[22] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, “A Survey on Zero Trust Architecture: Challenges and Future Trends,” Wireless Communications and Mobile Computing, vol. 2022, Art. no. 6476274, 2022, doi: 10.1155/2022/6476274.

[23] S. Li, M. Iqbal, and N. Saxena, “Future Industry Internet of Things with Zero-trust Security,” Information Systems Frontiers, vol. 26, no. 5, pp. 1653–1666, 2024, doi: 10.1007/s10796-021-10199-5.

[24] Microsoft, “Why is my Azure for Students subscription disabled and how do I reactivate it?,” Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/azurestudents-subscription-disabled

[25] Amazon Web Services, “AWS Free Tier FAQs,” 2026. [Online]. Available: https://aws.amazon.com/free/faqs/

Glossary and Abbreviations

Abbreviation	Full Term	Meaning
ACL	Access Control List	A set of rules controlling network traffic.
AI	Artificial Intelligence	Computational methods used to perform tasks associated with intelligent behaviour.
AWS	Amazon Web Services	Public cloud platform used for the AWS workload environment.
Azure	Microsoft Azure	Cloud platform used for the monitoring, function and model-storage components.
CIDR	Classless Inter-Domain Routing	Notation used to define IP address ranges and subnet boundaries.
CLI	Command Line Interface	Text-based interface for operating cloud services.

DLP	Data Loss Prevention	Controls intended to identify or restrict inappropriate data exposure.
EC2	Elastic Compute Cloud	AWS virtual compute service used for the project workloads.
IAM	Identity and Access Management	Controls for identities, authentication and authorisation.
IaC	Infrastructure as Code	Management of infrastructure through version-controlled configuration.
KQL	Kusto Query Language	Query language used in Microsoft Sentinel/Log Analytics.
ML	Machine Learning	Methods that learn patterns from data for prediction or anomaly detection.
NACL	Network Access Control List	Subnet-level AWS traffic filtering control.
PAM	Privileged Access Management	Controls for privileged administrative access.
SASE	Secure Access Service Edge	Architecture combining networking and security capabilities.
SDN	Software-Defined Networking	Policy-driven approach to network configuration and control.
SIEM	Security Information and Event Management	Centralised security telemetry, analysis and alerting platform.
ZTNA	Zero Trust Network Access	Identity- and policy-based access to applications and resources.

Table 7: Glossary and Abbreviations

Appendices

Appendix I: Industry Client Information

Industry Client Information

Project Title: Secure Cloud Access and Network Segmentation for a Hybrid Enterprise

Name of Industry Placement Agent (if applicable):

Agent Name: Arif Systems

Contact Name: Kate

Contact Number: NA
Contact Email: info@arifsystems.org

Industry Client Information (Mandatory for all proposals):

Company Name: ThinkiX
ABN:
Company Address:
Company Profile:
Website:

Industry Professional Contact Name: Tharindu Athauda
Client Number: +61 420 533 956
Client Email: tharindu@thinkix.com.au

Project Definition

Background and Rationale for the Project:

ThinkiX is a Melbourne-based software services company operating in a hybrid IT environment with on-premises infrastructure and public cloud services. With the rapid adoption of cloud-based collaboration tools and remote work, employees access enterprise resources from multiple locations and devices. This has increased the organisation's attack surface, exposing internal systems to risks such as unauthorised access, lateral movement, and data breaches. To address these challenges, ThinkiX proposes implementing a Zero Trust–based network architecture using secure access gateways, identity-aware firewalls, and network segmentation. The project focuses on designing a secure networking model that enforces least-privilege access while maintaining operational efficiency.

Project Goals and Objectives:

The goal is to design and prototype a secure, scalable hybrid network architecture. Objectives include implementing network segmentation to isolate critical systems, enforcing identity-based access controls, reducing the risk of lateral movement within the network, and supporting secure remote access without traditional VPN dependency.

Desired Outcomes/Deliverables:

Zero Trust access policy documentation; network segmentation and firewall rule configuration; technical report and configuration artifacts.

Project Resources:

Firewall and identity management platforms; sample enterprise network data; supervisor guidance.

Signature of Client: Tharindu Athauda **Date:31/05/2026:**

Signature of Industry Assigned Supervisor: Dr.Sana Jokhio **Date:**

Appendix II: Weekly Attendance Record with Industry Client/Supervisor

Insert the completed and signed weekly attendance record provided by the client/supervisor.

Week	Meeting Date	Meeting Mode	Students Present	Client Signature / Date
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

Table 8: Attendance Record with Industry Client/Supervisor

Appendix III: Summary of Updates for the Group Report Assessment 1

Feedback / Improvement Area	Action Taken
Weekly activities and responsibilities	Added a structured 12-week implementation timeline and responsibility allocation.
Detailed project design / implementation overview	Expanded the architecture-to-implementation mapping and added a final implementation and monitoring-response flow.
Evidence of implementation	Added placeholders for cloud console, Terraform, Cloudflare, Sentinel, Azure Function, model storage and NACL response evidence.
AI-SIEM implementation maturity	Extended the earlier model-training work into a live Sentinel-query and Azure scoring workflow.
Testing clarity	Separated configuration evidence, functional tests, integration tests and controlled response tests.
Final evaluation	Updated the report to distinguish implemented controls from remaining production-scale capabilities.

Table 9: Summary of Updates

Appendix IV: Client and Supervisor Meeting Logbook

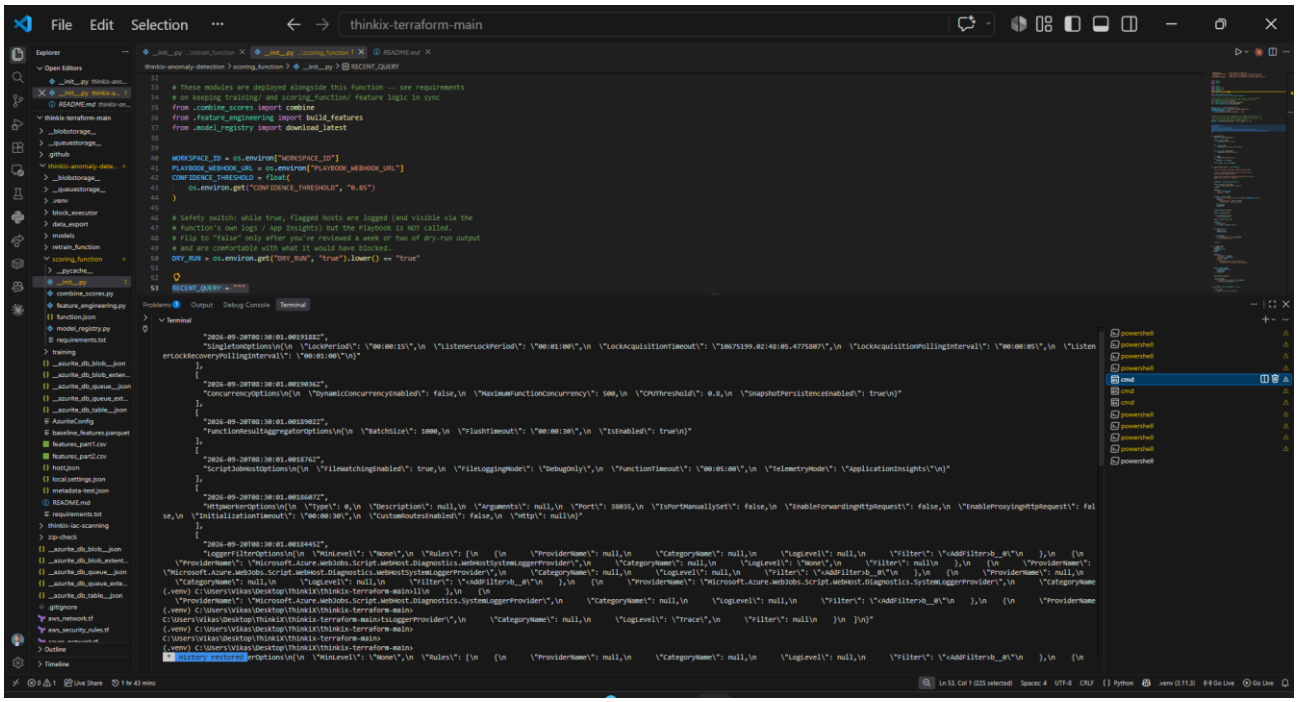
Insert the completed client and supervisor meeting logbooks with dates, attendees, agenda, minutes and actions.

Week	Meeting Information	Agenda	Meeting Minutes	Discussion and Action
1				
2				
3				
4				
5				
Week	**Meeting Information**	**Agenda**	**Meeting Minutes**	**Discussion and Action**
1				
2				
3				
4				
5				

Table 10: Client and Supervisor Meeting Logbook

Appendix V: Source Codes or Other Relevant Information

The final source-code package includes the Terraform infrastructure, Cloudflare configuration artefacts, Azure Function source, feature-engineering and model-serving modules, and response integration components. For the final submission, the repository/archive should be attached or linked according to the submission requirements.



Appendix V Figure: Final source-code structure

MELBOURNE
INSTITUTE OF TECHNOLOGY

Melbourne Campus

T +61 3 8600 6700

288 La Trobe Street,
Melbourne VIC 3000

Sydney Campus

T +61 2 8267 1400

154-158 Sussex Street,
Sydney, NSW 2000

mit.edu.au

CRICOS Provider Code 01545C and 03245K NSW TEQSA
Provider Code 12138